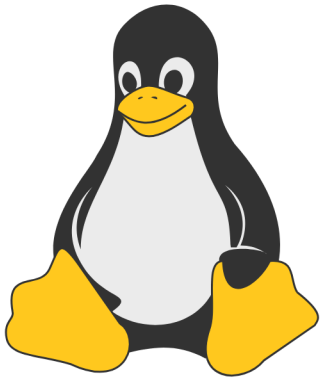
Linux for Security

A Practical Guide

- Master the Linux command line from the first login
- Users, permissions, processes, logs, and SSH
- Harden a real server, the base for every security tool





Contents

Section 1: Why Security People Use Linux 1

1. What Linux Is 2
2. Open Source, and Why It Matters for Security 3
3. Where Linux Runs: Servers, Devices, and the Cloud 4
4. Why Security Tools Live on Linux 5
5. Distributions, and Which to Use 5
6. The Command-Line Mindset 6
7. What This Course Covers 7

Section 2: Installing Linux in a Virtual Machine 11

1. Why a Virtual Machine 12
2. Choosing a Distribution: Ubuntu and Kali 13
3. Getting and Checking the Image 13
4. Creating the Virtual Machine 14
5. First Boot and Login 15
6. Snapshots: Your Safety Net 16
7. The First Update 16

Section 3: The Shell and the File System 20

1. What a Shell Is 21
2. The Prompt, and Your First Commands 22
3. The Linux File System Tree 22
4. Moving Around: pwd, cd, and ls 23
5. Absolute and Relative Paths 24
6. Your Home Directory, and Hidden Files 25
7. Getting Help: man and --help 25

Course mark icon from Material Symbols by Google, Apache License 2.0. Linux is a registered trademark of Linus Torvalds. Ubuntu is a trademark of Canonical Ltd. The Tux logo is by Larry Ewing and The GIMP. Used here only to refer to the software.
Cover photo: “Ta Prohm, Angkor 22” by LBM1948, CC BY-SA 4.0, via Wikimedia Commons. Cropped and toned.

1

Section 1

Why Security People Use Linux

IN THIS SECTION

1. What Linux Is 2

2. Open Source, and Why It Matters for Security 3

3. Where Linux Runs: Servers, Devices, and the Cloud 4

4. Why Security Tools Live on Linux 5

5. Distributions, and Which to Use 5

6. The Command-Line Mindset 6

7. What This Course Covers 7

What You Will Learn

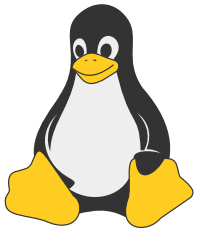
- What Linux is.
- What "open source" means, and why it matters for security.
- Where Linux runs: servers, the cloud, phones, and devices.
- Why almost every security tool lives on Linux.
- What a distribution is, and which ones you will use.
- The command-line mindset, and why security people prefer it.
- What this course covers.

Before You Start

You need Security Fundamentals (01) and How Computers Work (02). From course 02 you know what an operating system is, a kernel, processes, files, and the command line's existence. This lesson explains *why* Linux, in particular, is the operating system security people live in. There are no commands yet; the hands-on work starts in lesson 2, when you install Linux in a virtual machine (building directly on course 02's virtualization lesson).

---

1. What Linux Is



Linux is an operating system, like Windows or macOS (course 02, lesson 5): the software that manages a computer's hardware and runs its programs. The name technically refers to the **kernel**, the privileged core you met in course 02, written originally by Linus Torvalds in 1991 and developed since by thousands of people worldwide. In everyday use, "Linux" means the whole operating system built around that kernel.

Two things make Linux different from Windows and macOS, and both matter enormously for security:

- **It is free and open source** (topic 2): anyone can get it, use it, study it, and change it, at no cost.
- **It is built around the command line** (topic 6): while Linux has graphical desktops, its heart and its power are in text commands typed into a shell.

You may rarely see Linux on a home laptop, where Windows and macOS dominate. But step behind the scenes of the internet, and Linux is everywhere: it runs most of the world's servers, most of the cloud, the Android phones in billions of pockets, and countless devices. For a security person, Linux is not an alternative to learn someday; it is the ground you will stand on every day.

2. Open Source, and Why It Matters for Security

Open source means the source code (course 02, lesson 8) is public: anyone can read it, check it, and modify it. Linux is the most famous open-source project in the world. This is the opposite of **closed source** (or proprietary) software like Windows, whose code is a company secret.

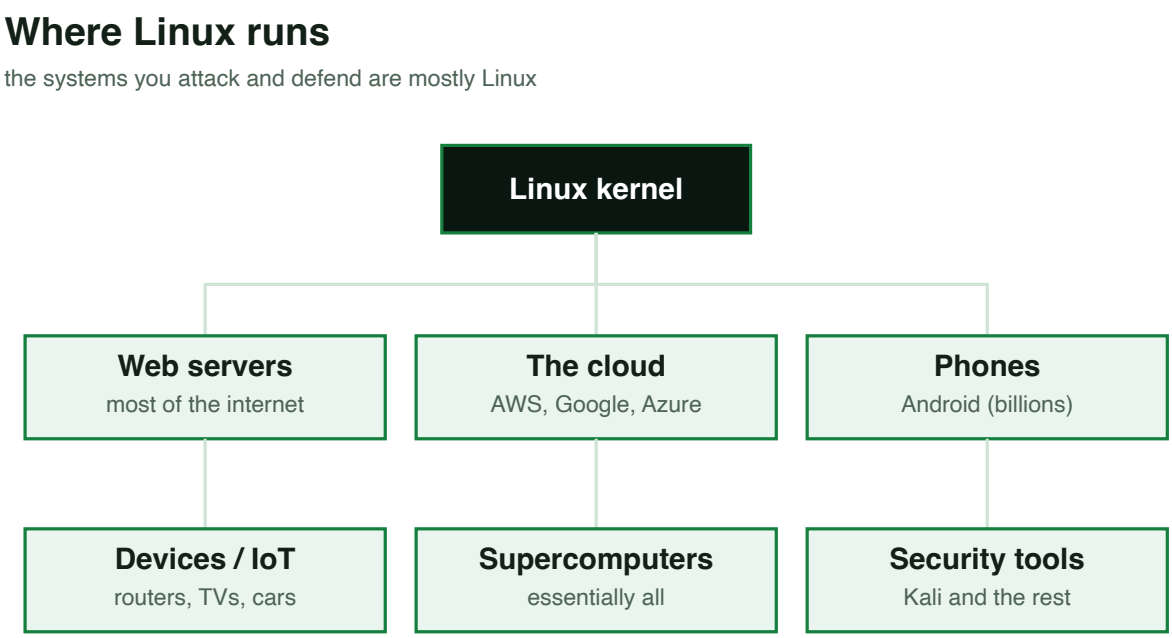
Why does this matter so much for security?

- **Anyone can inspect it.** Thousands of experts read Linux's code, so flaws and backdoors are far harder to hide. "Many eyes" is not a guarantee of safety, open-source software has bugs too, but secret code cannot be independently checked at all. You trust closed software on the vendor's word; you can *verify* open software.
- **You can see exactly what it does.** For a security person, this is priceless. You can read how a tool works, study how an attack functions, and understand the system to its core, instead of treating it as a black box.
- **It is free and modifiable.** Security researchers build, change, and share tools freely. The entire ecosystem of security tools (the next topics) grew because the foundation was open.
- **It teaches transparency.** The security field values understanding over mystery, and open source embodies that: nothing is hidden, everything can be examined. This matches the whole spirit of How Computers Work, a computer is not magic; you can look inside.

Open source is a big reason the security community adopted Linux: a field built on understanding systems naturally prefers a system you are allowed to understand completely.

3. Where Linux Runs: Servers, Devices, and the Cloud

If Linux is rare on home laptops, where is it? Nearly everywhere else that matters:



Where	How much Linux
Web servers	The large majority of the world's servers run Linux
The cloud	Most cloud computers (AWS, Google Cloud, Azure VMs) run Linux
Phones	Android, on billions of phones, is built on the Linux kernel
Devices (IoT)	Routers, smart TVs, cameras, cars, and countless gadgets run Linux
Supercomputers	Essentially all of the world's fastest computers run Linux

Think about what this means for security. **The systems attackers target, and that defenders protect, are mostly Linux.** When you investigate a hacked web server, it is almost certainly Linux. When you secure a company's cloud, you are securing Linux machines. When you analyze a compromised router or IoT device, it runs Linux. A security professional who cannot work on Linux is locked out of most of the real targets and most of the real defense.

This is why Linux comes so early in this program, right after the fundamentals. Almost every later course, offensive, defensive, cloud, assumes you can work on a Linux system. This course makes sure you can.

4. Why Security Tools Live on Linux

Beyond being the systems you attack and defend, Linux is also where security *tools* live. The overwhelming majority of security software, the scanners, the packet analyzers, the exploitation frameworks, the forensic kits, is built for Linux, and much of it runs only on Linux.

There are good reasons:

- **Linux gives you deep control.** Security work often needs low-level access, to raw network packets, to processes, to the file system, that Linux grants more readily than a locked-down consumer OS.
- **It is scriptable and automatable.** The command line (topic 6) lets you combine tools and automate tasks, which is the heart of efficient security work (and the reason the Python and scripting courses exist).
- **It is free, so tools and labs cost nothing.** You can run dozens of Linux machines in your lab without licensing fees.
- **The community builds there.** Because the foundation is open and the experts are already there, new tools appear on Linux first.

There is even a Linux distribution made specifically for offensive security, **Kali Linux**, which comes pre-loaded with hundreds of security tools. You will use it as your "attacker machine" in the home lab (course 08) and the pentesting courses. For now, the point is simple: **to use the tools of the trade, you must be comfortable on Linux.** This course builds that comfort.

5. Distributions, and Which to Use

Here is something that surprises newcomers: there is no single "Linux" you download. Because Linux is open source, many groups package the kernel together with different sets of programs, settings, and looks, to make a complete, ready-to-use operating system. Each of these packaged versions is called a **distribution** (or **distro**).

All distributions share the same Linux kernel, so skills transfer between them, but they differ in their default tools, their package manager (lesson 7), and their focus. A few you will meet:

Distribution	Known for
Ubuntu	Friendly, popular, well-documented; great for learning and for servers
Debian	Stable and minimal; Ubuntu is built on it
Kali Linux	Pre-loaded with offensive security tools; the attacker's distro
Red Hat / CentOS / Rocky	Common in large companies and enterprise servers

This course uses **Ubuntu**, because it is beginner-friendly, widely used on real servers, and well-documented, so what you learn applies directly to the real world. You will install it in the next lesson. You will meet **Kali** later, when you need its attack tools. Do not worry about choosing a distro yourself yet; learn the fundamentals on Ubuntu, and because the skills transfer, you will be able to work on any of them.

6. The Command-Line Mindset

The biggest shift for a newcomer is this: on Linux, security work is done mostly by **typing commands**, not clicking buttons. Linux has graphical desktops, but the command line, the **shell** (lesson 3), is where the real power is. This feels harder at first, and then becomes far faster and more powerful. Embracing it is the single most important mindset change this course asks of you.

Why do security people prefer the command line?

- **It is precise.** A command does exactly what you say, with no hidden behavior, which matters when the stakes are high.
- **It is powerful.** Small commands combine into pipelines (lesson 8) that do in one line what would take many clicks, or that no graphical tool offers at all.
- **It is repeatable and automatable.** A command can be saved in a script and run again, on one machine or a thousand. You cannot automate clicking.
- **It works everywhere.** A server has no screen or mouse; you reach it over the network (SSH, lesson 11) and work entirely by command. The command line is the *only* option on most of the systems you will touch.
- **It shows you the machine.** Commands reveal exactly what is happening, which fits the "a computer is not magic" view this program builds.

Do not expect to love it on day one. Expect it to feel slow and strange, then, around the time you build your first pipeline, to click, and to start feeling clumsy whenever you have to use a mouse instead. That shift is the goal.

7. What This Course Covers

This course takes you from your first Linux login to hardening a real server. The path:

Part	Lessons	What you will be able to do
Getting in	1–4	Install Linux, move around the shell and file system, work with files
Users and processes	5–7	Manage users and permissions, control processes and services, install software
Text, network, logs	8–10	Build command pipelines, use networking commands, read logs
Remote and hardening	11–13	Use SSH, harden a server, and build a hardened web server

Everything is hands-on and happens in **your own Ubuntu virtual machine**, which you will build in the next lesson. Because the VM is yours, the one rule from Security Fundamentals, only test what you own, is satisfied by construction, and snapshots (course 02's virtualization lesson) mean any mistake is undone in seconds.

By the end, you will be comfortable on the command line, able to manage and secure a Linux system, and ready for every course that follows, because nearly all of them live on Linux. The next lesson gets you there: installing Linux in a virtual machine, your safe, personal practice system.

Common Mistakes

1. Thinking Linux is only for experts or servers. It runs most of the internet, the cloud, and Android. For security, it is the main ground, not a niche.

2. Believing open source is less secure "because anyone can see the code". The opposite argument is at least as strong: anyone can also *find and fix* flaws, and you can verify it rather than trust a vendor's word.

3. Expecting one "Linux" to download. There are many distributions sharing one kernel. This course uses Ubuntu; skills transfer to all of them.

4. Avoiding the command line. It is the whole point. Servers have no mouse, and the command line is more precise, powerful, and automatable. Lean into it.

5. Thinking you must pick the "best" distro now. Learn the fundamentals on Ubuntu; because skills transfer, you can use any distro later, including Kali for attacks.

Summary

Idea	Remember
Linux	A free, open-source operating system; its kernel runs the systems that matter
Open source	Public code anyone can inspect and change; you can verify, not just trust
Where it runs	Most servers, the cloud, Android, and devices, the systems you attack and defend
Security tools	Almost all live on Linux (including Kali, the attacker's distro)
Distribution	A packaged version of Linux; this course uses Ubuntu; skills transfer
Command-line mindset	Precise, powerful, automatable, and the only option on most systems
This course	From first login to hardening a server, all in your own Ubuntu VM

Check Your Understanding

1. What, technically, is "Linux", and what does the word mean in everyday use?
2. Give two reasons open source matters for security.
3. Name three places Linux runs, and say why that matters for a security person.
4. Why do almost all security tools run on Linux?
5. What is a distribution, and which does this course use and why?
6. Give three reasons security people prefer the command line.
7. Why is it safe to do everything in this course without risking a real system?

Answers

1. Technically, Linux is the kernel, the privileged core of the OS, written originally by Linus Torvalds. In everyday use, "Linux" means the whole operating system built around that kernel.
2. Any two: anyone can inspect the code, so flaws and backdoors are harder to hide and you can verify rather than trust; you can see exactly what software does; it's free and

modifiable, enabling the security-tool ecosystem; it teaches transparency.

3. Any three: web servers, the cloud, Android phones, IoT devices, supercomputers. It matters because the systems attackers target and defenders protect are mostly Linux.
4. Because Linux gives deep low-level control, is scriptable and automatable, is free (so tools and labs cost nothing), and is where the open community builds, so tools appear there first.
5. A distribution is a packaged version of Linux (the kernel plus programs, settings, and a look). This course uses Ubuntu, because it's beginner-friendly, widely used on servers, and well-documented; skills transfer to other distros.
6. Any three: it's precise (does exactly what you say); powerful (pipelines do more than clicks); repeatable/automatable (saved in scripts, run anywhere); works everywhere (servers have no screen); and shows you the machine.
7. Because everything runs in your own virtual machine, which you own (satisfying "only test what you own"), and snapshots let you undo any mistake in seconds.

Practice

No computer needed yet, just reflect and write.

1. List three devices or services you used today that are likely running Linux behind the scenes (think servers, phones, routers).
2. In your own words, explain to a friend why a security person needs Linux, using the fact that most servers and the cloud run it.
3. Write down one worry you have about using the command line, then one reason from this lesson why it is worth pushing through that worry. (Keep it; revisit it at the end of the course.)

A Worked Example

1. My Android phone (Linux kernel); any website I visited (its server is almost certainly Linux); my home Wi-Fi router (most run Linux).
2. "Most of the world's web servers and cloud computers run Linux, so the systems a security person has to protect, or investigate after a hack, are mostly Linux. If you can't work on Linux, you're locked out of most real security work. It's not optional, it's the main ground."
3. Worry: "Typing commands feels slower and I'm afraid of breaking something." Reason to push through: commands are more precise, powerful, and automatable than clicking, and

they're the only way to work on servers, which have no mouse, plus my VM's snapshots mean mistakes are undone in seconds.

2

Section 2

Installing Linux in a Virtual Machine

IN THIS SECTION

1. Why a Virtual Machine 12
2. Choosing a Distribution: Ubuntu and Kali 13
3. Getting and Checking the Image 13
4. Creating the Virtual Machine 14
5. First Boot and Login 15
6. Snapshots: Your Safety Net 16
7. The First Update 16

What You Will Learn

- Why you install Linux in a virtual machine, not on your real computer.
- How to choose a distribution for this course.
- How to get the Linux image and check it is genuine.
- How to create the virtual machine.
- How to boot and log in for the first time.
- How to take a snapshot, your safety net.
- How to run the first system update.

Before You Start

You need How Computers Work (02), especially its virtualization lesson, you know what a virtual machine, a hypervisor, and a snapshot are. This lesson turns that knowledge into a real, working Linux system you own. It is mostly steps to follow on your own computer. By the end you will have an Ubuntu virtual machine, the practice system for the rest of this course. Set aside about an hour; the download is the slow part.

---

1. Why a Virtual Machine

You will not install Linux directly onto your laptop. Instead you will run it inside a **virtual machine (VM)**, a full simulated computer running on top of your real one (How Computers Work, lesson 10). Everything in this course happens inside that VM.

Why a VM, and not your real machine?

- **Safety.** You will run commands that can break a system, change permissions, delete files, edit configuration. In a VM, a mistake harms only the VM, never your real computer and its data.
- **Snapshots.** A VM can be saved and restored in seconds (topic 6). Break something badly? Restore a snapshot and you are instantly back to a clean state. This makes learning fearless.
- **It keeps your real computer clean.** No changes to your laptop; when you are done, you can delete the VM and nothing remains.
- **It satisfies the one rule.** The VM is *yours*. Everything you do, including the attacks you will practise in later courses, happens on a system you own, so the "only test what you own"

rule (Security Fundamentals, lesson 9) is met by construction.

This VM is the single practice machine for this course. (The full home lab, with multiple machines on an isolated network, comes in course 08; here you need just one.) You run it with a **hypervisor**, the software that creates and runs VMs, which you install first.

2. Choosing a Distribution: Ubuntu and Kali

From lesson 1, a **distribution** is a packaged version of Linux, and this course uses **Ubuntu**: beginner-friendly, widely used on real servers, and well-documented, so your skills apply to the real world. Specifically, use the current **Ubuntu LTS** ("Long-Term Support") desktop release, which is stable and supported for years.

You will also hear about **Kali Linux**, the distribution pre-loaded with offensive security tools. You do *not* need Kali for this course; Ubuntu is the right choice for learning Linux fundamentals. You will install Kali later, as your attacker machine, when the home lab (course 08) and the pentesting courses need it. For now: **one Ubuntu VM.**

Pick a hypervisor for your computer:

- **VirtualBox** (free, Windows/Linux, and Intel Macs) — the common choice.
- **UTM** (free, Apple Silicon Macs) — because VirtualBox support on Apple Silicon is limited.
- **VMware** (free for personal use) — another solid option.

Install the hypervisor for your system first, from its official site (see the sources), the same way you install any application.

3. Getting and Checking the Image

Next, download the Ubuntu **image**: a single file (an `.iso`) containing the installer, from the official Ubuntu website only. Downloading an operating system from an unofficial site is a real risk: a tampered image could contain malware, and you would be installing it yourself. This is the supply-chain idea from Security Fundamentals, made concrete.

So, two security habits from the very start:

- **Download only from the official source** (`ubuntu.com`). Never from a random mirror, link, or torrent you do not trust.
- **Check the image is genuine.** Official downloads come with a **checksum** (a hash, How Computers Work lesson 2's fingerprint idea, studied fully in Cryptography): a long value that uniquely identifies the correct file. After downloading, you compute the hash of your

file and compare it to the official value. If they match, your file is the real, unmodified image. If not, do not use it.

You do not need to master hashing yet; just know the habit, verify what you download, especially something as powerful as an operating system. Ubuntu's download page explains how to check the hash for your system.

4. Creating the Virtual Machine

With the hypervisor installed and the Ubuntu image downloaded, create the VM. The exact clicks differ by hypervisor, but the steps are the same everywhere:

1. **Create a new virtual machine** and point it at the Ubuntu `.iso` you downloaded.
2. **Give it resources.** A reasonable starting point: 2 CPU cores, 4 GB of RAM, and 25 GB of disk. (Recall from How Computers Work what each means.) You can adjust later.
3. **Start the VM.** It boots from the installer image, exactly as a real computer boots from a USB stick (the boot process, How Computers Work lesson 9).

Screenshot to capture

The new-VM settings screen with the Ubuntu ISO selected and 2 CPU / 4 GB RAM / 25 GB disk set.

Mac/PC, your hypervisor

`assets/screenshots/03-linux/03-linux/vm-create-settings.png`

The VM is a computer-within-a-computer: it has its own virtual disk, memory, and network, all isolated from your real machine (How Computers Work lesson 10). When it starts, you are watching a fresh computer boot for the first time.

5. First Boot and Login

The VM boots into the Ubuntu **installer**. Follow its prompts; the defaults are sensible for learning:

1. Choose your language and keyboard.
2. Choose a normal installation.
3. When asked about the disk, let it use the whole (virtual) disk, remember, this is the VM's virtual disk, not your real one, so "erase disk" here is completely safe.
4. **Create your user account.** Use the name **`dara`** (this course's examples all use `dara`), pick a password you will remember, and set the computer's name to **`ubuntu`**. Your prompt will then read `dara@ubuntu`, matching every example in this course.
5. Let the installer finish and restart the VM.

Screenshot to capture

The "Who are you?" screen with name dara and computer name ubuntu filled in.
the Ubuntu installer

`assets/screenshots/03-linux/03-linux/ubuntu-create-account.png`

After it restarts, you log in with your password and reach the Ubuntu desktop. Open the **Terminal** application, the window where you type commands, and you will see your first prompt:

```
dara@ubuntu:~$
```

That prompt is where you will live for the rest of this course. The next lesson explains exactly what every part of it means. You now have a real, working Linux machine.

6. Snapshots: Your Safety Net

Before you do anything else, learn the most important safety habit for a VM: the **snapshot** (How Computers Work, lesson 10). A snapshot saves the VM's entire state so you can return to it instantly.

Right now, you have a clean, freshly installed Ubuntu. **Take a snapshot and name it something like "clean install".** In your hypervisor, this is a menu option (often "Take Snapshot" or "Snapshots"). It takes a few seconds.

Why do this immediately, and often?

- **Mistakes become free.** This course has you run powerful commands. If you ever break your VM, delete the wrong files, misconfigure something, lock yourself out, you simply **restore the snapshot** and you are back to clean in seconds, instead of reinstalling.
- **You can experiment boldly.** Snapshots are what let you try things without fear, which is how you learn fastest.
- **It is a real security practice.** Analysts snapshot a VM before detonating malware, then restore it afterward (Malware Analysis, course 29). You are building the habit now.

Get into the rhythm: **snapshot before anything risky, and whenever your VM is in a known-good state.** It is the single habit that makes learning Linux (and later, handling malware) safe.

7. The First Update

One real task before you finish, and it is a security task. A freshly installed system is already somewhat out of date, because fixes have been released since the image was built. So the first thing to do on any new Linux system is **update it**, applying the latest security patches (the "updates are a security control" idea from Security Fundamentals and How Computers Work).

In the terminal, you run two commands (you will learn exactly how they work in lesson 7 on packages; for now, just run them):

```
dara@ubuntu:~$ sudo apt update
dara@ubuntu:~$ sudo apt upgrade
```

- `sudo` runs the command with administrator power (you will meet it properly in lesson 5); it will ask for your password.
- `apt update` refreshes the list of available software and updates.
- `apt upgrade` downloads and installs the updates. Type `y` to confirm when asked.

When it finishes, your system has the latest security fixes. **Take another snapshot now** (name it "updated"), so you have a clean, fully updated starting point to return to.

You have done a great deal: you understand why a VM is the right place to learn, you have installed Ubuntu, logged in, taken your safety-net snapshots, and applied the first security updates. You have a real Linux system that is yours to explore and impossible to permanently break. In the next lesson, we start using it properly: the shell, the prompt, and the Linux file system.

Common Mistakes

1. Installing Linux on your real computer instead of a VM. You will run commands that can break a system. A VM contains any mistake; your real machine and data stay safe.

2. Downloading Ubuntu from an unofficial site. An operating system from an untrusted source could be tampered with. Use the official site, and check the hash.

3. Skipping snapshots. Snapshots make mistakes free and learning fearless. Take one right after install, after updating, and before anything risky.

4. Panicking about "erase disk" in the installer. It means the VM's *virtual* disk, which is empty and isolated. Your real disk is untouched.

5. Not updating a fresh install. A new system is already behind on security fixes. Update it first, every time, on every new Linux system.

Summary

Step	Remember
Use a VM	Safe, snapshot-able, isolated, and yours (satisfies "only test what you own")
Distribution	Ubuntu LTS desktop for this course; Kali comes later as the attacker machine
Get the image	Official source only; verify the checksum (a hash)
Create the VM	Point it at the ISO; ~2 CPU, 4 GB RAM, 25 GB disk; boot it
Account	User `dara`, computer `ubuntu`, so your prompt matches the course
Snapshot	Save a clean state; restore in seconds; do it before anything risky
First update	`sudo apt update && sudo apt upgrade`, then snapshot again

Check Your Understanding

1. Give two reasons you install Linux in a VM instead of on your real computer.
2. Which distribution does this course use, and why not Kali?
3. Why must you download the Ubuntu image only from the official site, and what does checking the checksum prove?
4. In the installer, why is "erase disk" safe here?
5. What user name and computer name does the course use, and why does that matter?
6. What does a snapshot let you do, and when should you take one?
7. Why is updating a freshly installed system a security task, and which commands do it?

Answers

1. Any two: safety (mistakes harm only the VM); snapshots (restore a clean state in seconds); it keeps your real computer clean; it satisfies "only test what you own" because the VM is yours.
2. Ubuntu (LTS desktop). Not Kali because Ubuntu is beginner-friendly and right for learning fundamentals; Kali's attack tools come later, in the lab and pentesting courses.
3. Because an OS from an untrusted source could be tampered with (a supply-chain risk); checking the checksum (hash) proves the file matches the official, unmodified image.
4. Because it refers to the VM's virtual disk, which is empty and isolated from your real machine, so nothing real is erased.
5. User `dara` and computer `ubuntu`, so the prompt reads `dara@ubuntu` and matches every example in the course, avoiding confusion.
6. It saves the VM's entire state so you can restore it instantly. Take one after install, after updating, and before anything risky.
7. Because a fresh image is already behind on security fixes released since it was built, so updating applies the latest patches (a security control). The commands are `sudo apt update` then `sudo apt upgrade`.

Practice

This practice is the lesson: set up your VM.

1. Install a hypervisor (VirtualBox, UTM, or VMware) for your computer, and download Ubuntu LTS from the official site. Check the checksum if you can.
2. Create a VM (about 2 CPU, 4 GB RAM, 25 GB disk), install Ubuntu with user `dara` and computer name `ubuntu`, and log in. Open the Terminal and confirm your prompt reads

`dara@ubuntu:~$`.

3. Take a snapshot named "clean install". Then run `sudo apt update` and `sudo apt upgrade`, and take a second snapshot named "updated".

Write one sentence noting what each snapshot captures, so you know which to restore to later.

A Worked Example

A completed setup: VirtualBox installed on a PC (or UTM on an Apple Silicon Mac); Ubuntu LTS downloaded from ubuntu.com and its SHA-256 checksum matched the official value; a VM with 2 CPUs, 4 GB RAM, and a 25 GB virtual disk; Ubuntu installed with user `dara` on computer `ubuntu`; the Terminal showing `dara@ubuntu:~$`; a snapshot "clean install" taken right after install, then `sudo apt update && sudo apt upgrade` run, then a snapshot "updated". Restore "updated" to return to a patched, clean system; restore "clean install" to go all the way back.

3

Section 3

The Shell and the File System

IN THIS SECTION

1. What a Shell Is 21

2. The Prompt, and Your First Commands 22

3. The Linux File System Tree 22

4. Moving Around: pwd, cd, and ls 23

5. Absolute and Relative Paths 24

6. Your Home Directory, and Hidden Files 25

7. Getting Help: man and --help 25

What You Will Learn

- What a shell is.
- What the prompt means, and your first commands.
- The Linux file system tree.
- Moving around with `pwd`, `cd`, and `ls`.
- Absolute and relative paths.
- Your home directory, and hidden files.
- Getting help with `man` and `--help`.

Before You Start

You need lesson 2: a working Ubuntu VM with the `dara` account, and the Terminal open. Everything here is typed into that terminal. This is where the hands-on course truly begins. Every command is shown exactly as it runs, with the prompt `dara@ubuntu:~$`. Type each one yourself, nothing here can harm your VM, and if you ever worry, you have your snapshots.

---

1. What a Shell Is

When you open the Terminal, you are talking to a **shell**: a program that reads the commands you type, runs them, and shows you the result. The shell is the layer between you and the operating system (course 02, lesson 5): you type a command, the shell asks the kernel to do it, and the output comes back to your screen. The default shell on most Linux systems, including Ubuntu, is called **Bash**.

Think of the shell as a conversation with the computer, one line at a time:

- You type a **command** and press Enter.
- The shell runs it (often by starting a program, course 02's processes).
- The result, the **output**, appears below.
- The shell shows a fresh **prompt**, ready for your next command.

This loop, type, run, read, repeat, is how all command-line work happens, from this first lesson to the most advanced attack in later courses. The shell feels bare at first, but it is the most direct, honest way to use a computer: you say exactly what you want, and you see exactly what happens. That is why security people live in it.

2. The Prompt, and Your First Commands

Before you type anything, the shell shows a **prompt**. On your Ubuntu VM it looks like this:

```
dara@ubuntu:~$
```

Every part of it tells you something:

- **`dara`** is your username (course 02, lesson 5: who you are).
- **`ubuntu`** is the computer's name (the one you set during install).
- **`~`** is where you are right now in the file system, `~` is shorthand for your home directory (topic 6).
- **`$`** means you are a normal user. (A `#` here would mean you are root, the all-powerful account, lesson 5, a sign to be careful.)

So `dara@ubuntu:~$` reads as "the user dara, on the computer ubuntu, in the home directory, as a normal user." This small line answers the two questions every security person asks on a system: *who am I* and *where am I* (course 02's "explore a running system" project).

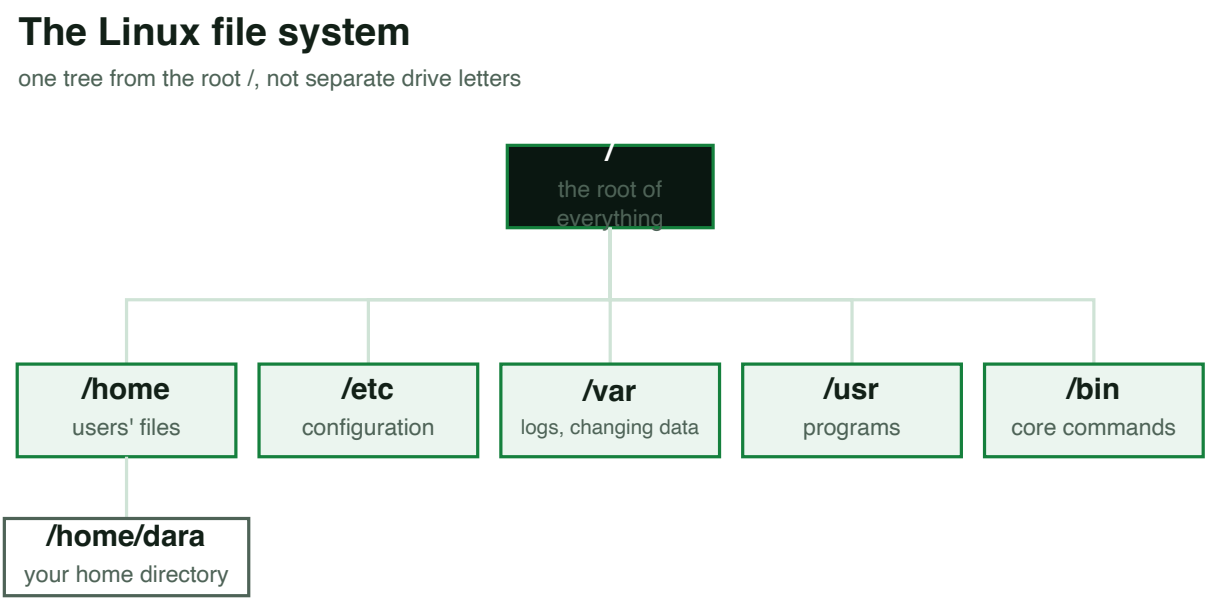
Try your first command, `whoami`, which prints your username:

```
dara@ubuntu:~$ whoami
dara
```

The shell ran the command and printed the answer, `dara`, then showed a new prompt. That is the whole rhythm of the command line. Everything from here is more of the same: a command, its output, a new prompt.

3. The Linux File System Tree

Before moving around, you need the map. In course 02 (lesson 7) you learned that files live in directories forming a tree. Linux organizes everything into **one single tree**, starting from a top called the **root**, written as a single slash `/`. There are no separate drive letters like Windows' `C:`; every disk, every file, every device hangs somewhere off that one `/`.



A few important directories, which you will meet again and again:

Directory	What it holds	Why security cares
`/`	The root of everything	The top of the whole tree
`/home`	Each user's personal files (`/home/dara`)	Where user data lives
`/etc`	System configuration files	How the system and services are set up
`/var`	Data that changes, especially **logs** (`/var/log`)	Where evidence of activity is kept (lesson 10)
`/usr` and `/bin`	Installed programs and core commands	What the system can run

You do not need to memorize all of these now. Just hold the shape: **one tree, rooted at `/`, with `/home` for users, `/etc` for configuration, and `/var/log` for logs.** When you investigate a Linux system later (forensics, incident response), you will go straight to these places.

4. Moving Around: pwd, cd, and ls

Three commands let you move through that tree and see what is around you. They are the first commands you will use constantly.

`pwd` ("print working directory") tells you where you are:

```
dara@ubuntu:~$ pwd
/home/dara
```

So `~` in the prompt was `/home/dara`, your home directory. **`ls`** ("list") shows what is in the current directory:

```
dara@ubuntu:~$ ls
Desktop  Documents  Downloads  Pictures  Public  Videos
```

These are the standard folders in a fresh home directory. **`cd`** ("change directory") moves you somewhere else:

```
dara@ubuntu:~$ cd Documents
dara@ubuntu:~/Documents$ pwd
/home/dara/Documents
```

Notice the prompt changed from `~` to `~/Documents`, the shell always shows where you are. To go back up one level to the parent directory, use `cd ..` (the `..` from course 02, lesson 7):

```
dara@ubuntu:~/Documents$ cd ..
dara@ubuntu:~$
```

And `cd` on its own, with nothing after it, always takes you home. With just these three, `pwd` to see where you are, `ls` to see what is here, `cd` to move, you can navigate the entire system. This is the ground floor of every Linux session.

5. Absolute and Relative Paths

When you tell a command where something is, you give a **path** (course 02, lesson 7). There are two kinds, and the difference matters.

- An **absolute path** starts from the root `/` and names the full route, so it works from anywhere. Example: `/home/dara/Documents`.
- A **relative path** starts from where you are *now*. Example: if you are in `/home/dara`, then `Documents` (relative) means the same place as `/home/dara/Documents` (absolute).

Both of these do the same thing if you start in your home directory:

```
dara@ubuntu:~$ cd /home/dara/Documents      # absolute: works from anywhere
dara@ubuntu:~/Documents$ cd                 # back home
dara@ubuntu:~$ cd Documents                 # relative: from here
dara@ubuntu:~/Documents$
```

Two shortcuts appear in relative paths, both from course 02: `.` means "here" (the current directory) and `..` means "up one level." So `cd ..` goes up, and `./something` means "something right here."

This matters for security because, as course 02's files lesson noted, the `..` trick lets a careless program be pushed *out* of where it should stay (**path traversal**). Understanding absolute versus relative paths, and what `..` does, is the foundation for both using Linux and understanding that whole family of attacks (Web Application Security, course 11).

6. Your Home Directory, and Hidden Files

Your **home directory** (`/home/dara`, shown as `~`) is your own space: where your files live and where you can freely create, edit, and delete, without affecting the rest of the system. As a normal user, you have full control here and limited power elsewhere, that is least privilege (Security Fundamentals, lesson 7) built into the system.

But `ls` did not show everything. Linux has **hidden files**: any file whose name starts with a dot (`.`) is hidden from a normal listing. These are usually configuration files, and they are hidden just to keep listings tidy, not for security. To see them, add the `-a` flag ("all") to `ls`:

```
dara@ubuntu:~$ ls -a
.  ..  .bashrc  .cache  .config  .profile  Desktop  Documents  Downloads
```

Now you see more: `.` (this directory) and `..` (the parent), and several dot-files like `.bashrc` (settings for your shell) and `.config` (where programs store their settings).

This is an important lesson for security: **"hidden" does not mean secret or safe.** Hidden files are trivially visible with `ls -a`. Attackers often hide malicious files and configuration by giving them dot-names, hoping a casual look misses them, so an investigator always lists hidden files. When you examine a system, `ls -a` is a reflex: never trust a plain `ls` to show you everything.

7. Getting Help: man and --help

You will never memorize every command and option, and you do not need to, because Linux documents itself. Two tools give you help on the spot.

`man` ("manual") opens the full manual page for a command. For example, `man ls` shows everything `ls` can do:

```
dara@ubuntu:~$ man ls
LS(1)                    User Commands                    LS(1)

NAME
       ls - list directory contents

SYNOPSIS
       ls [OPTION]... [FILE]...

DESCRIPTION
       List information about the FILEs (the current
       directory by default)...
```

(Press `q` to quit the manual.) The manual tells you what a command does, and every flag it accepts. Learning to read `man` pages is one of the most valuable Linux skills; it makes you self-sufficient.

For a quicker reminder, most commands accept **`--help`**, which prints a short summary:

```
dara@ubuntu:~$ ls --help
Usage: ls [OPTION]... [FILE]...
List information about the FILEs (the current directory by default).
...
  -a, --all      do not ignore entries starting with .
  -l             use a long listing format
...
```

There is the `-a` flag you just used, and `-l` for the long listing you saw in lesson 2. Between `man` (full detail) and `--help` (a quick reminder), you can always find out what a command does and how to use it, which is exactly how real Linux users and security professionals work: not by memorizing, but by knowing how to look it up.

You now have the foundation: you understand the shell and the prompt, the shape of the file system, how to move around and list files (including hidden ones), the difference between absolute and relative paths, and how to get help. In the next lesson, you start working *with* files: creating, viewing, editing, and finding them.

Common Mistakes

1. Not reading the prompt. `dara@ubuntu:~$` tells you who you are, where you are, and whether you are root (`#`). Glancing at it prevents many errors.

2. Confusing absolute and relative paths. An absolute path starts with `/` and works anywhere; a relative path depends on where you are. Mixing them up sends commands to the wrong place.

3. Trusting a plain `ls`. It hides dot-files. Attackers use dot-names to hide. Use `ls -a` to see everything, as a reflex.

4. Thinking "hidden" means secret. Hidden files are visible instantly with `ls -a`. Hiding is not protecting (an echo of course 02's encoding-vs-encryption point).

5. Trying to memorize every command. You cannot, and you do not need to. Use `man` and `--help`; knowing how to look things up is the real skill.

Summary

Command / idea	Remember
Shell (Bash)	Reads commands, runs them, shows output; the type-run-read loop
The prompt `dara@ubuntu:~$`	user @ computer : location, and `$` (normal) vs `#` (root)
File system tree	One tree from root `/`; `/home`, `/etc`, `/var/log` matter most
`pwd` / `ls` / `cd`	Where am I / what's here / move; `cd ..` up, `cd` home
Absolute vs relative path	From `/` (anywhere) vs from here; `.` here, `..` up
Home and hidden files	`~` = `/home/dara`; `ls -a` shows dot-files ("hidden" ≠ secret)
`man` / `--help`	Full manual / quick summary; look things up, don't memorize

Check Your Understanding

1. What does a shell do, and what is the default shell on Ubuntu?
2. In the prompt `dara@ubuntu:~$`, what do `dara`, `ubuntu`, `~`, and `$` each mean?
3. What do `pwd`, `ls`, and `cd` each do?
4. What is the difference between an absolute and a relative path? Give an example of each for the Documents folder.
5. What does `ls -a` show that `ls` does not, and why does that matter in security?
6. Is a "hidden" (dot) file secret? Why or why not?
7. You meet a command you do not know. Name two ways to find out what it does.

Answers

1. A shell reads the commands you type, runs them (asking the kernel), and shows the output. Ubuntu's default shell is Bash.
2. `dara` = your username; `ubuntu` = the computer's name; `~` = your current location (home directory); `$` = you are a normal user (a `#` would mean root).
3. `pwd` prints where you are (the working directory); `ls` lists what is in the current directory; `cd` changes to another directory (`cd ..` goes up, `cd` alone goes home).
4. An absolute path starts from `/` and works from anywhere; a relative path starts from where you are now. For Documents: absolute `/home/dara/Documents`; relative (from home) `Documents`.
5. `ls -a` also shows hidden files (names starting with a dot). It matters because attackers hide malicious files with dot-names, so an investigator must list hidden files to see everything.
6. No. A dot-file is hidden only from a plain listing and is revealed instantly by `ls -a`. Hiding is not protecting.
7. `man <command>` (the full manual page) and `<command> --help` (a quick summary).

Practice

In your Ubuntu VM's terminal:

1. Run `whoami`, `pwd`, and `ls`. Write down who you are, where you are, and what is in your home directory.
2. Use `cd` to go into `Documents`, run `pwd` to confirm, then `cd ..` to go back, and `cd` alone to make sure you are home. Watch how the prompt changes each time.
3. Run `ls` and then `ls -a`. How many more items does `-a` show, and what are two of them?
4. Run `man ls`, read the first screen, and find what the `-l` flag does. Press `q` to quit.

A Worked Example

1. `whoami` → `dara`. `pwd` → `/home/dara`. `ls` → `Desktop Documents Downloads Pictures Public Videos`.
2. `cd Documents` then `pwd` → `/home/dara/Documents` (prompt shows `~/Documents`); `cd ..` returns to `/home/dara` (prompt `~`); `cd` keeps you at home.
3. `ls` shows the six folders; `ls -a` adds several dot-items, for example `.`, `..`, `.bashrc`, and `.config`, the hidden files a plain `ls` leaves out.

4. In `man ls`, the `-l` entry reads "use a long listing format", the detailed listing (permissions, owner, size, date) you saw in lesson 2. Press `q` to return to the prompt.